

LoRaWAN Capacity & Scaling

Technical Reference for a Single Network Server

How many gateways can one ChirpStack handle?

How many devices? Where do the limits live?

What knobs turn when growth pushes the system?

Version 1.0 | June 2026 | Kigali, Rwanda

1. The Mental Model

Most people ask this question backwards. They start with 'how many devices can ChirpStack handle?' — but the network server is almost never the binding constraint. The radio link is.

A LoRaWAN system has three independent capacity layers. The one you hit first depends on what your network is doing — not on raw compute power.

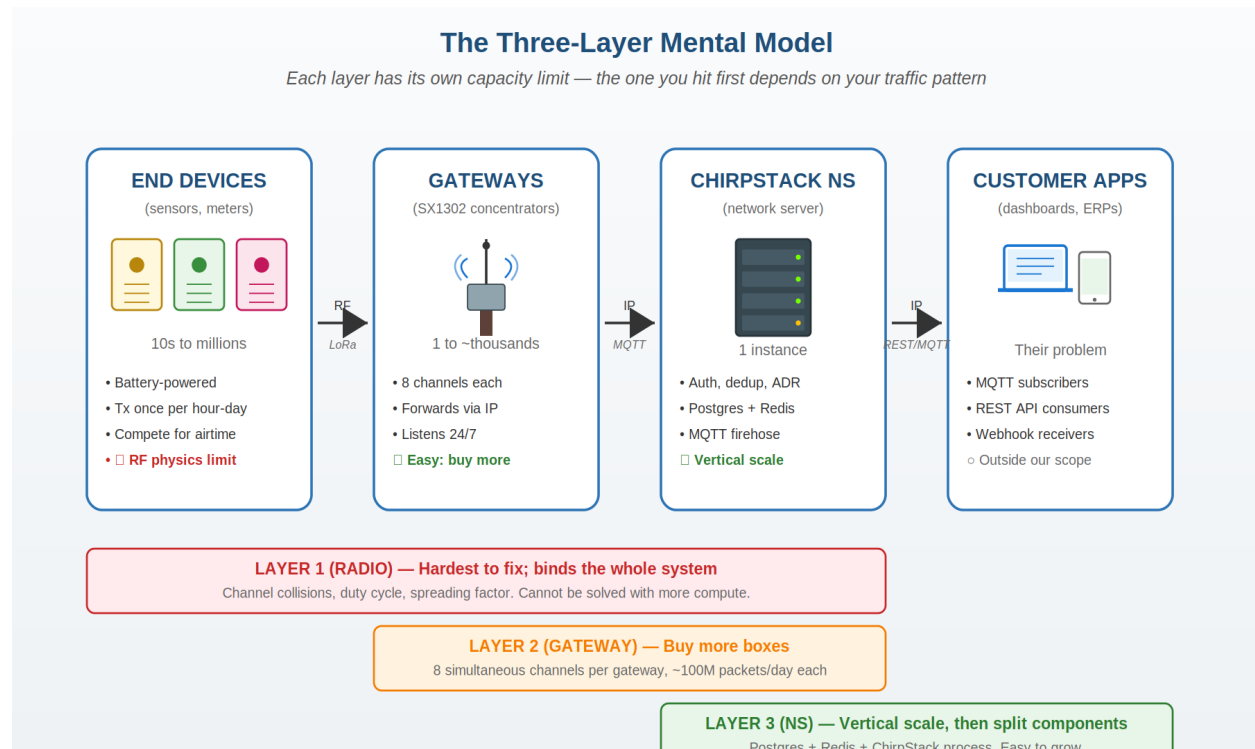


Figure 1 — The three-layer model: radio, gateway hardware, and network server. Each scales differently.

The headline numbers

Layer	Limit	How to scale
Radio (RF)	~30 packets/sec per gateway channel before collisions explode	More gateways (denser coverage), lower SF (ADR), fewer uplinks per device
Gateway hardware	8 simultaneous channels per gateway, ~10,000 typical devices	Buy more gateways
Network server (NS)	One €5/mo VPS holds thousands of devices comfortably	Bigger VPS, then split components, then horizontal scale
Customer apps	Their problem, not ours	Customer must throttle their consumption

⚠ Critical principle: Radio is the binding limit, not the server. One ChirpStack instance can theoretically handle 18-36 million devices according to published benchmarks. You will run out of radio airtime long before you run out of CPU.

2. Layer 1 — Radio (RF) Capacity

This is the layer that physically constrains every LoRaWAN deployment. It's a function of three things: spreading factor (SF), duty cycle regulation, and the underlying ALOHA contention protocol that LoRaWAN uses.

2.1 The spreading factor trade-off

LoRa uses different spreading factors (SF7 through SF12). Higher SF means longer range but exponentially longer airtime. This is the single most important radio-layer concept to understand.

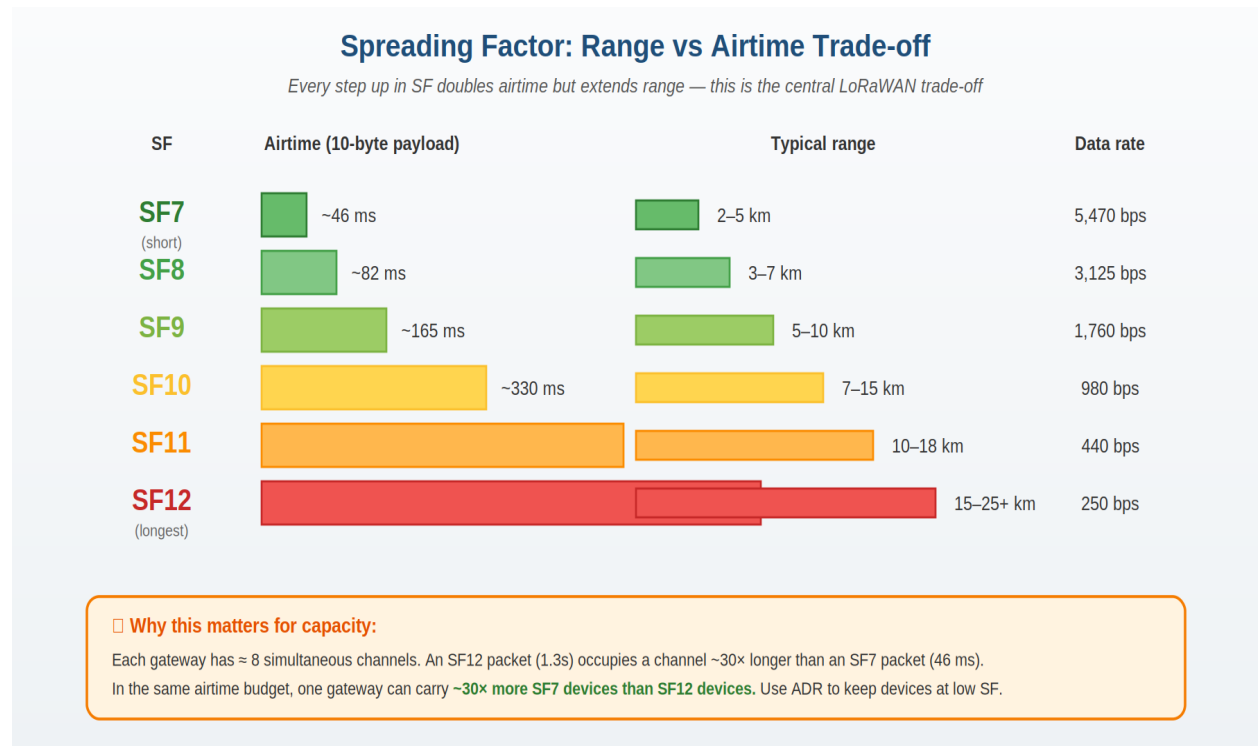


Figure 2 — As SF increases by one step, airtime roughly doubles. A single SF12 packet occupies a channel ~30× longer than SF7.

2.2 Duty cycle regulation

In EU868 (the band we deploy in Rwanda for production), each device is legally limited to 1% duty cycle per sub-band. This means a single device transmitting at SF12 (1.3s airtime) can only transmit once every ~130 seconds — that's its regulatory ceiling, not a network limit.

In US915 (our bench setup), the rule is different: 400ms max dwell time per channel, no overall duty cycle. The result is similar capacity in practice.

2.3 ALOHA collision physics

LoRaWAN uses pure ALOHA: devices transmit whenever they want, with no coordination. From queueing theory, the maximum theoretical throughput of a pure ALOHA channel is ~18% utilisation. Above that, collisions dominate and effective throughput collapses.

Practical rule: keep average channel utilisation under 10% to leave headroom. Above ~20%, you lose more than half of all packets to collisions.

3. Layer 2 — Gateway Capacity

Modern SX1302-based gateways (what we deploy) listen on 8 channels simultaneously across all spreading factors. This is wildly more than older SX1301 designs.

3.1 Per-gateway theoretical capacity

The simple formula:

$$\text{max_devices} \approx (3600 \times 8 \times 0.10) / (\text{uplinks_per_hour} \times \text{avg_airtime_seconds})$$

Where 3600 is seconds-per-hour, 8 is the channel count, and 0.10 is the safe ALOHA utilisation factor.

3.2 Real-world capacity by use case

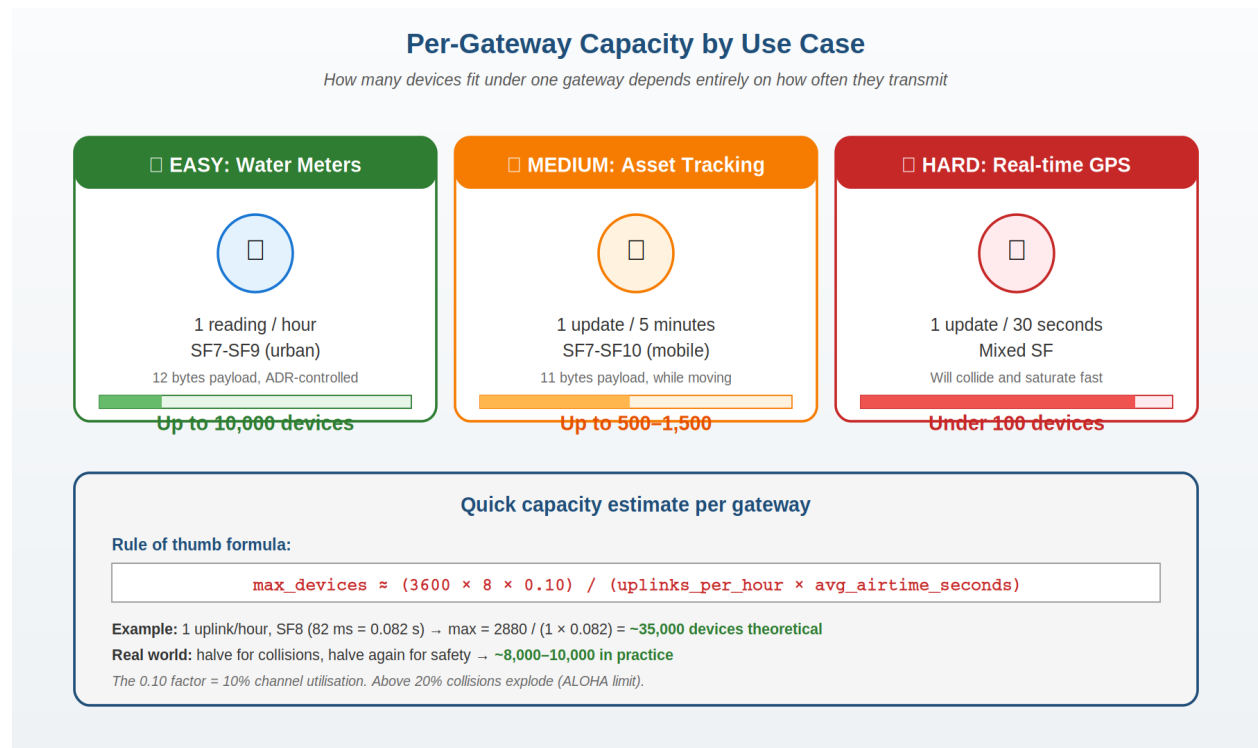


Figure 3 — The same gateway can carry 10,000 water meters or under 100 real-time GPS trackers. Use case dictates capacity.

3.3 Per-gateway numbers by traffic pattern

Use case	Uplinks per device	SF range	Devices per gateway
Water/electricity meters	1 per hour	SF7-SF9	5,000–10,000
Soil moisture / agriculture	4 per hour	SF7-SF10	1,500–3,000
Asset tracking (low frequency)	1 per 15 min	SF7-SF10	500–1,500
Cold-chain monitoring	6 per hour	SF7-SF9	1,000–2,500

Use case	Uplinks per device	SF range	Devices per gateway
Real-time GPS (1 per 30s)	120 per hour	SF7-SF10	Under 100
Smart parking	Event-driven	SF7-SF8	2,000–5,000

3.4 Why 'just buy more gateways' works

- Gateway hardware is commoditised. A RAK7268CV2 is ~USD 350; a Seeed WM1302 on Raspberry Pi is ~USD 150.
- Two gateways in the same area DO NOT halve coverage of a single device — they add redundancy. ChirpStack receives the packet from both gateways and deduplicates.
- Coverage overlap is desirable. It gives ADR more data points to optimise each device's spreading factor.
- Each additional gateway adds 8 channels of independent capacity for new devices in its coverage area.

 **Density principle:** Five gateways in a 3 km² area serve more devices than fifty gateways scattered thinly across 50 km². Cover one city densely before expanding to the next.

4. Layer 3 — Network Server (NS) Capacity

This is where most people overspend. The published ChirpStack benchmarks show one process handling 18,000 uplinks per second. At one uplink per hour per device, that's 65 million devices — far more than we'll ever realistically connect.

4.1 Anatomy of a ChirpStack deployment

Five processes run on the network server. They can all live on one VPS, or be split for resilience as scale grows.

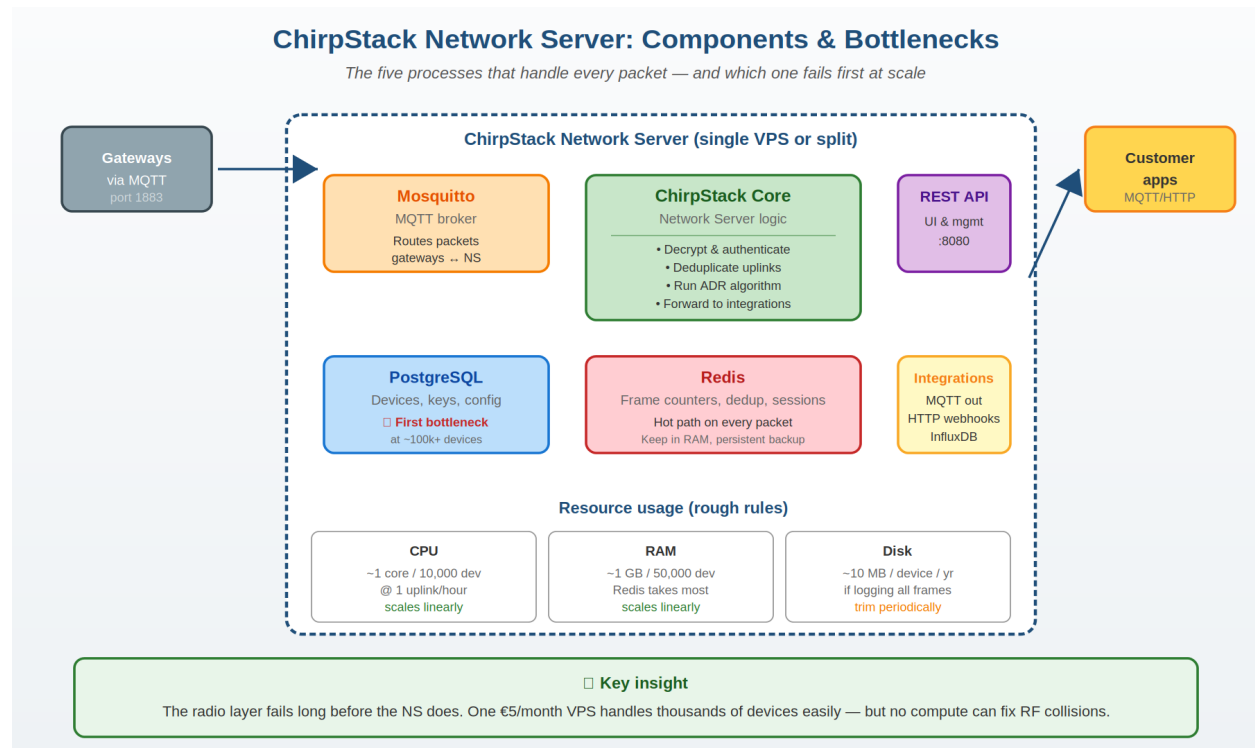


Figure 4 — Components of a ChirpStack deployment. Each packet passes through Mosquitto → ChirpStack Core → Redis lookup → Postgres query → Integrations.

4.2 Resource consumption rules of thumb

Resource	Consumption rate	Behaviour
CPU	~1 core per 10,000 devices at 1 uplink/hour	Scales linearly. Mostly used by ChirpStack core + Postgres queries.
RAM	~1 GB per 50,000 devices	Redis takes most of it (session state, dedup cache, frame counters).
Disk	~10 MB per device per year if logging all frames	Trim or off-load periodically. Postgres grows fastest.
Network bandwidth	Negligible — LoRaWAN payloads are 12-50 bytes typical	Even 100k devices is well under 1 Mbit/s sustained.

Resource	Consumption rate	Behaviour
MQTT throughput	Mosquitto easily handles 100k+ msg/sec on modest hardware	Almost never the bottleneck.

4.3 What fails first as you grow

In order of likelihood:

- **PostgreSQL slowness** (at ~100k+ devices) — Solution: bigger instance, then read replicas.
- **Redis memory pressure** (at ~500k+ devices) — Solution: more RAM, then Redis Cluster.
- **ChirpStack core CPU** (at very high uplink rates) — Solution: split components into separate VPSs.
- **Disk space** (at ~5+ years of frame logs) — Solution: archive old data to object storage.

5. VPS Sizing Tiers

Here's what the network looks like at every realistic scale. Each tier is a clean migration from the previous — same software stack, just bigger box.

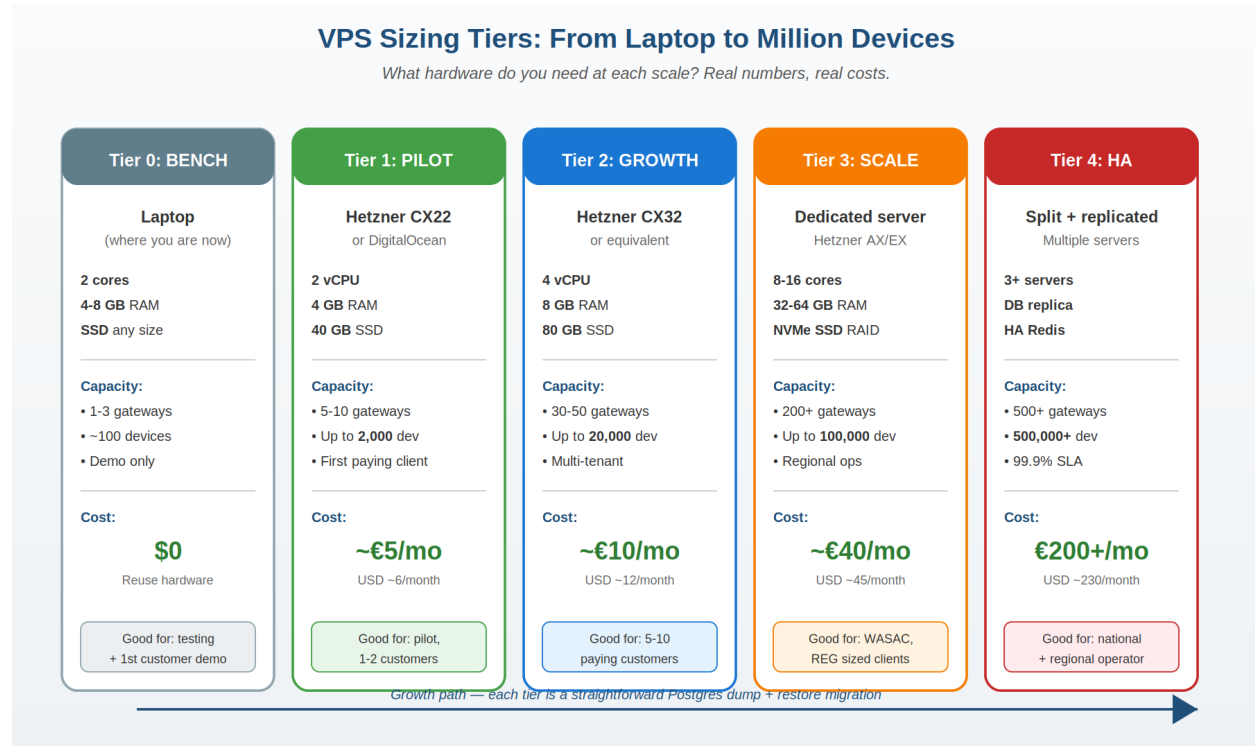


Figure 5 — VPS sizing tiers. Migration between tiers is a Postgres dump-and-restore plus DNS update.

5.1 Tier explanations

Tier 0 — Bench (where we are now)

- Your laptop with Docker Compose. Free.
- Good for: developing the integration, testing devices, demoing to customers.
- NOT good for production — laptop power loss = customer outage.

Tier 1 — Pilot (€5/month)

- Hetzner CX22 or DigitalOcean equivalent: 2 vCPU, 4 GB RAM, 40 GB SSD.
- Capacity: 5-10 gateways, up to 2,000 devices.
- Move here as soon as you have your first paying customer.
- Migration from Tier 0: ~2 hours of focused work.

Tier 2 — Growth (€10/month)

- Hetzner CX32: 4 vCPU, 8 GB RAM, 80 GB SSD.
- Capacity: 30-50 gateways, up to 20,000 devices.
- This tier covers most realistic Year-2 ambitions for the Rwanda network.

Tier 3 — Scale (€40/month)

- Dedicated server (Hetzner AX or EX line): 8-16 cores, 32-64 GB RAM, NVMe RAID.
- Capacity: 200+ gateways, up to 100,000 devices.
- Move here once WASAC or similar Tier-A customer is live in production.

Tier 4 — High Availability (€200+/month)

- Multiple servers with split components, replicated database, HA Redis.
- Capacity: 500+ gateways, 500,000+ devices.
- 99.9% uptime SLA territory. Move here when you need it, not before.

6. ADR — The Most Important Lever

Adaptive Data Rate (ADR) is built into the LoRaWAN protocol. ChirpStack runs the ADR algorithm on the network server side, telling each device what SF and power to use based on observed link quality.

ADR is the single biggest capacity multiplier in LoRaWAN. Without it, every device transmits at SF12 (worst case) — destroying capacity. With it, each device uses the lowest SF that still works reliably.

ADR (Adaptive Data Rate): Why It's the Most Important Lever

The network server tells each device to use the lowest SF that still works — multiplying capacity ~5×

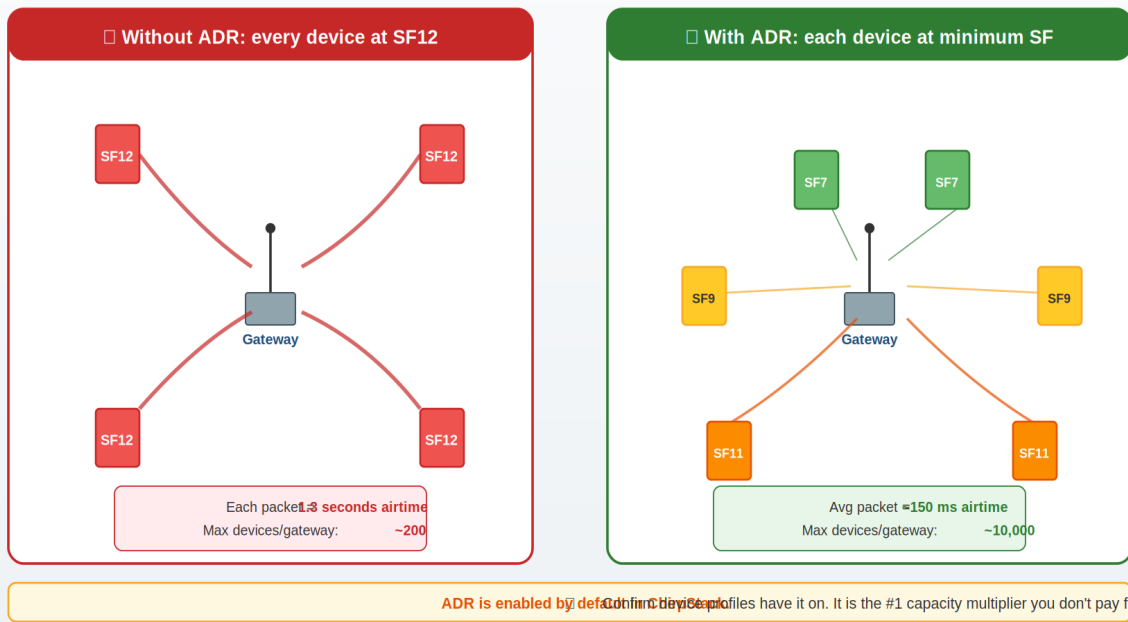


Figure 6 — Same gateway, same physical devices, with and without ADR. The capacity gain is 5× to 50×.

6.1 How ADR works in practice

- Device joins the network at SF12 (the safest, longest-range setting).
- After ~20 successful uplinks, the network server analyses RSSI and SNR.
- If the link has margin, NS sends an ADR command telling the device to step down to a lower SF (and potentially lower TX power).
- Result: a device 200m from a gateway will stabilise at SF7, while one 4km away stays at SF10.

6.2 Verifying ADR is on

In ChirpStack web UI:

- Device profiles → [your profile] → Make sure 'Adaptive data-rate' is **ON**
- Confirm the ADR algorithm is set to 'Default' unless you have a reason to override.

★ **Action item:** Verify ADR is enabled on all device profiles in ChirpStack. This costs nothing and increases per-gateway capacity by 5-50×. It is the single highest-leverage setting in the entire system.

7. Gateway Deployment Strategy

Where to put gateways, and in what order, matters more than how many.

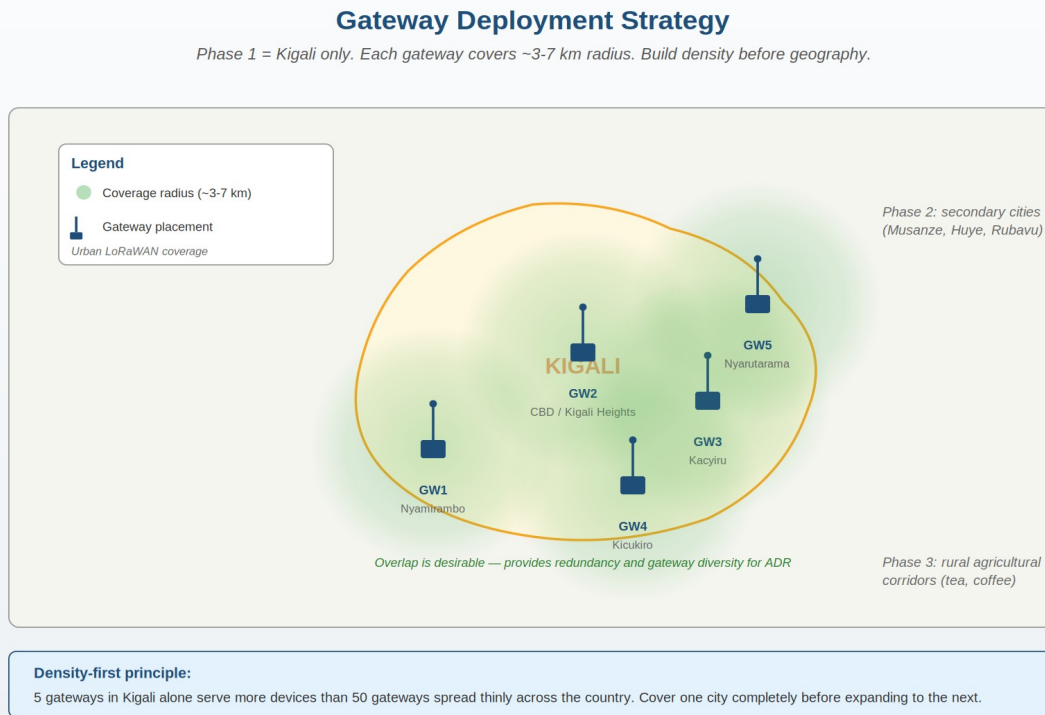


Figure 7 — Phase 1 gateway plan for Kigali. Overlap is desirable, not wasteful.

7.1 Phase 1 — Kigali (Month 0-12)

- 5 gateways covering the main districts: Nyamirambo, CBD, Kacyiru, Kicukiro, Nyarutarama.
- Indoor: rooftop or upper-floor placement. SMA dipole antenna outside the window minimum.
- Coverage: ~3 km radius dense urban, up to 7 km line-of-sight.
- Total capacity: easily 30,000+ devices in Kigali alone with this footprint.

7.2 Phase 2 — Secondary cities (Year 2)

- Musanze (Northern Province) — agricultural cooperatives + tourism.
- Huye (Southern) — university town, ag research.
- Rubavu (Western) — Lake Kivu, tea and tourism.
- One gateway per city as anchor, second added as customers grow.

7.3 Phase 3 — Rural corridors (Year 3+)

- Tea plantation corridors (Sorwathe / Mountain Tea).
- Coffee washing station areas.
- Specific to anchor customer needs — usually funded as part of the project, not speculative.

8. Load Management — Knobs to Turn

When you start to feel capacity pressure, here are the levers in order of effectiveness:

8.1 The capacity dial (in order of leverage)

Lever	Effect	Cost	Risk
1. Enable / verify ADR	5-50× more devices	Free	None
2. Reduce uplink frequency	Linear improvement	Free (firmware update)	Customer may notice delayed alerts
3. Reduce payload size	10-30% improvement	Free (firmware update)	Less data per packet
4. Add a gateway	Adds 8 channels of capacity	USD 150-500	Need installation site
5. Confined-area sub-band split	2-4× capacity in dense areas	Configuration	Devices must support sub-band selection
6. Upgrade VPS tier	More CPU, RAM, disk	USD 5-200/month	Brief outage during migration
7. Split ChirpStack components	Eliminates single-point bottleneck	More cloud spend	More operational complexity

8.2 Customer-side load management

Some load problems come from how customers consume the data, not how devices generate it. When you sell LoRaWAN-as-a-Service, set these expectations explicitly:

- Customers should consume MQTT through their own broker subscription, not by polling REST. Polling kills throughput.
- For dashboards: aggregate at the application layer (5-minute averages, not raw frames).
- For analytics: stream to a time-series database (InfluxDB, TimescaleDB), not the operational Postgres.
- Webhooks scale poorly — if a customer wants 100k devices' worth of webhooks, they need a queue.

9. Monitoring & Alerts

You can't manage capacity you can't measure. Here's the minimum monitoring you must have in production.

9.1 Per-gateway metrics

- **Uplink count per minute** — rising trend = approaching capacity.
- **Channel utilisation %** — alert at 15%, urgent action at 20%.
- **Last-seen timestamp** — alert if older than 5 minutes.
- **Packet loss / CRC error rate** — alert if above 5%.

9.2 Per-NS metrics

- **CPU load** — sustained > 70% triggers vertical scale planning.
- **Memory usage** — sustained > 80% means add RAM.
- **Postgres connection count** — close to max_connections = bottleneck imminent.
- **Redis hit ratio** — should stay > 95%, lower means cache too small.
- **Disk usage and growth rate** — forecast 90-day exhaustion early.

9.3 Recommended tooling stack

- Prometheus + Grafana — open source, runs on the same VPS, free dashboards
- UptimeRobot — free external uptime monitoring with SMS alerts
- ChirpStack's built-in stats dashboard — already gives you per-gateway health
- Postgres pg_stat_activity views — for query-level diagnostics

10. Quick Reference Cheat Sheet

10.1 If a customer asks 'how many devices?'

Ask them four questions:

- How often does each device transmit? (per hour / per day)
- How far from a gateway will devices be? (close / medium / far)
- How many gateways will be deployed?
- Is real-time critical or can data be delayed by minutes?

Then look up the answer in the table below.

10.2 Reference numbers per gateway

Configuration	Typical capacity
Water meters, 1 uplink/hour, ADR-on	5,000–10,000 devices
Soil sensors, 1 uplink/15 min, ADR-on	1,500–3,000 devices
Cold-chain, 1 uplink/10 min, ADR-on	1,000–2,000 devices
Asset tracker (mobile), 1 uplink/5 min	300–800 devices
High-frequency GPS, 1 uplink/30 sec	Under 100 devices

10.3 VPS sizing by device count

Device count	Recommended tier	Cost/month
< 500 devices, 1-3 gateways	Tier 0 (laptop) — bench only	\$0
500 — 2,000 devices	Tier 1 (Hetzner CX22)	€5
2,000 — 20,000 devices	Tier 2 (Hetzner CX32)	€10
20,000 — 100,000 devices	Tier 3 (Dedicated server)	€40
100,000+ devices	Tier 4 (Split + HA)	€200+

10.4 Decision flowchart in words

- **Are uplinks getting lost?** → Check ADR is on; check channel utilisation; if > 15%, add a gateway.
- **Is the web UI slow?** → Check Postgres CPU; if high, upgrade VPS tier or tune Postgres.
- **Are gateways going offline intermittently?** → Check internet connection on gateway, not the server.
- **Customer asks for more frequent data?** → Calculate the new airtime budget; add gateway if needed.
- **VPS at 70% CPU sustained?** → Migrate to next tier. Takes 2-4 hours including DNS propagation.

Appendix — Formulas & References

A. Quick airtime formula

$$T_{\text{packet}} \text{ (ms)} \approx 2^{\text{SF}} \times (8 + \max(0, \text{ceil}((8 \times \text{payload} - 4 \times \text{SF} + 28 + 16) / (4 \times (\text{SF} - 2\text{DE}))) \times \text{CR})) / \text{BW}$$

Where BW = bandwidth (125 kHz typical), DE = low-data-rate-optimisation flag, CR = coding-rate offset (4/5 to 4/8).

Don't try to memorise this. Use an online calculator (avbentem.github.io/airtime-calculator). The intuition that 'each SF step doubles airtime' is enough for capacity planning.

B. Sources

- **ChirpStack scalability benchmark (2024-2025)** — www.chirpstack.io/docs
- **Semtech AN1200.22** — LoRa Modulation Basics (the canonical reference for airtime calculations)
- **The Things Network Fair Use Policy** — widely-cited duty-cycle defaults used in commercial planning
- **ETSI EN 300.220** — European SRD regulations for the 868 MHz band
- **Bor et al. 2016** — Do LoRa Low-Power Wide-Area Networks Scale? — the foundational academic paper on LoRaWAN ALOHA collision behaviour

C. Glossary

- **ADR** — Adaptive Data Rate. NS-driven optimisation of device SF and TX power.
- **ALOHA** — Random-access protocol used by LoRaWAN. Max ~18% theoretical channel utilisation.
- **Airtime** — Time a single packet occupies the channel. Function of SF, payload size, bandwidth.
- **Duty cycle** — Percentage of time a transmitter may be on. EU868 = 1%.
- **SF** — Spreading Factor. Range 7-12. Higher = longer range, more airtime.
- **Sub-band** — A subset of channels within a regional plan. US915 has 8 sub-bands; we use sub-band 0.

End of document — LoRaWAN Capacity & Scaling Reference v1.0